

Apache Spark was hard until I learned these 30 concepts.

Six topics. Each concept is numbered where it first appears.

1 MapReduce, the problem Spark was built to fix counting words in a file



PROBLEM 1

Disk after every step

Each stage writes to HDFS, the next one reads it back. That round trip is what makes MapReduce slow.

PROBLEM 2

Only two steps

A filter, a join and an aggregation become several MapReduce jobs chained together.

What Spark changed

FIX 1

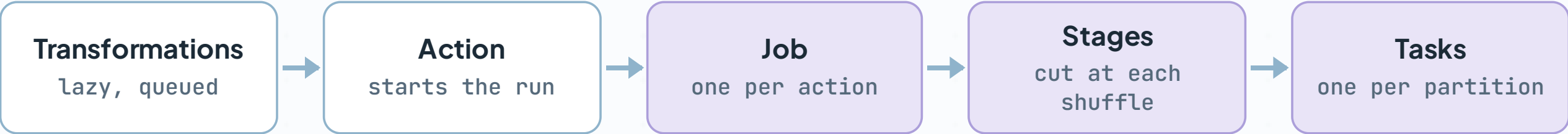
2 In-memory processing

Intermediate results stay in RAM across the nodes. No flush to disk between steps, so 10 to 100 times faster.

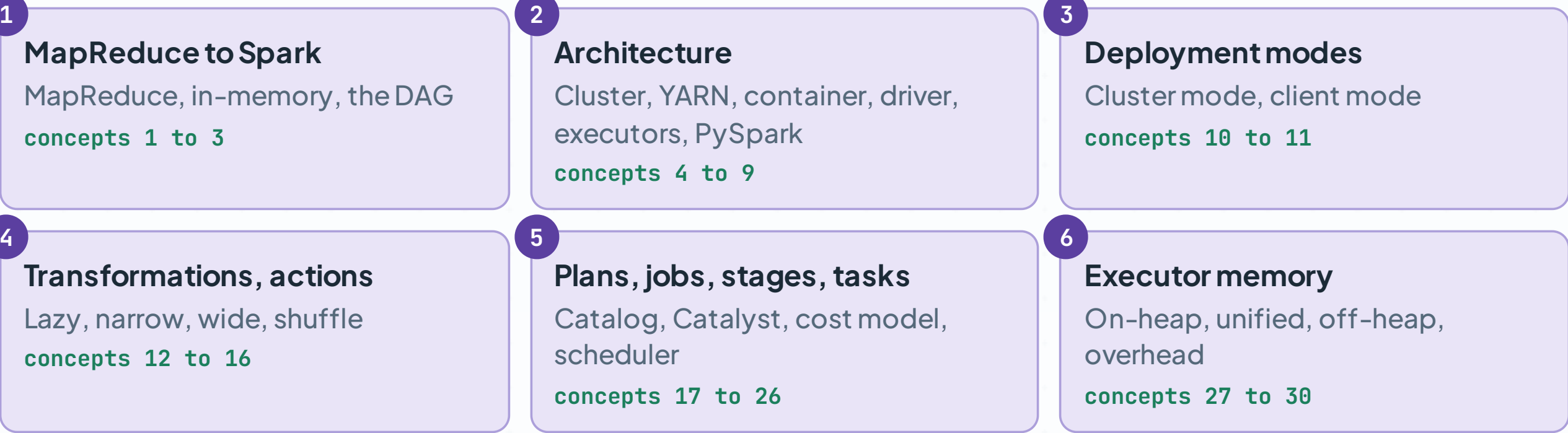
FIX 2

3 ADAG

Transformations and actions form a directed acyclic graph. Spark splits it into jobs, stages and tasks.



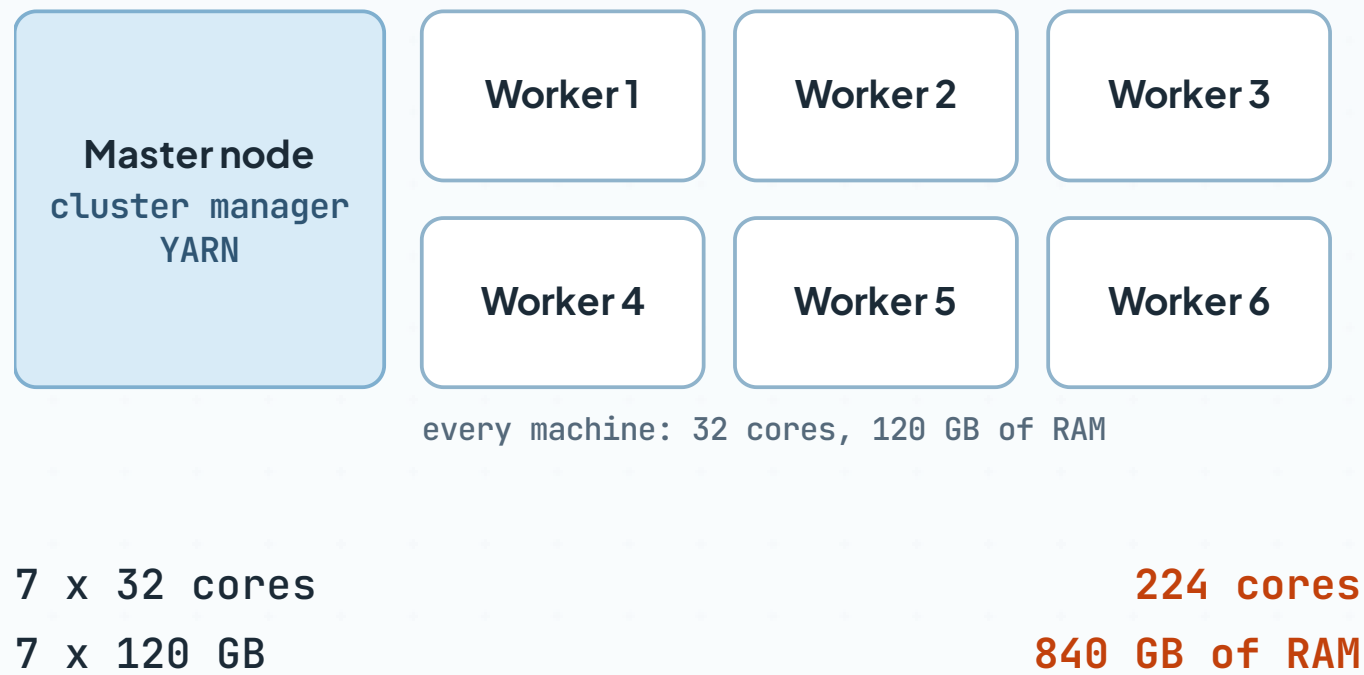
The 30 concepts, in six topics the green chips count them through the pages



What happens after spark-submit.

Concepts 4 to 9

The example cluster 7 machines, one network

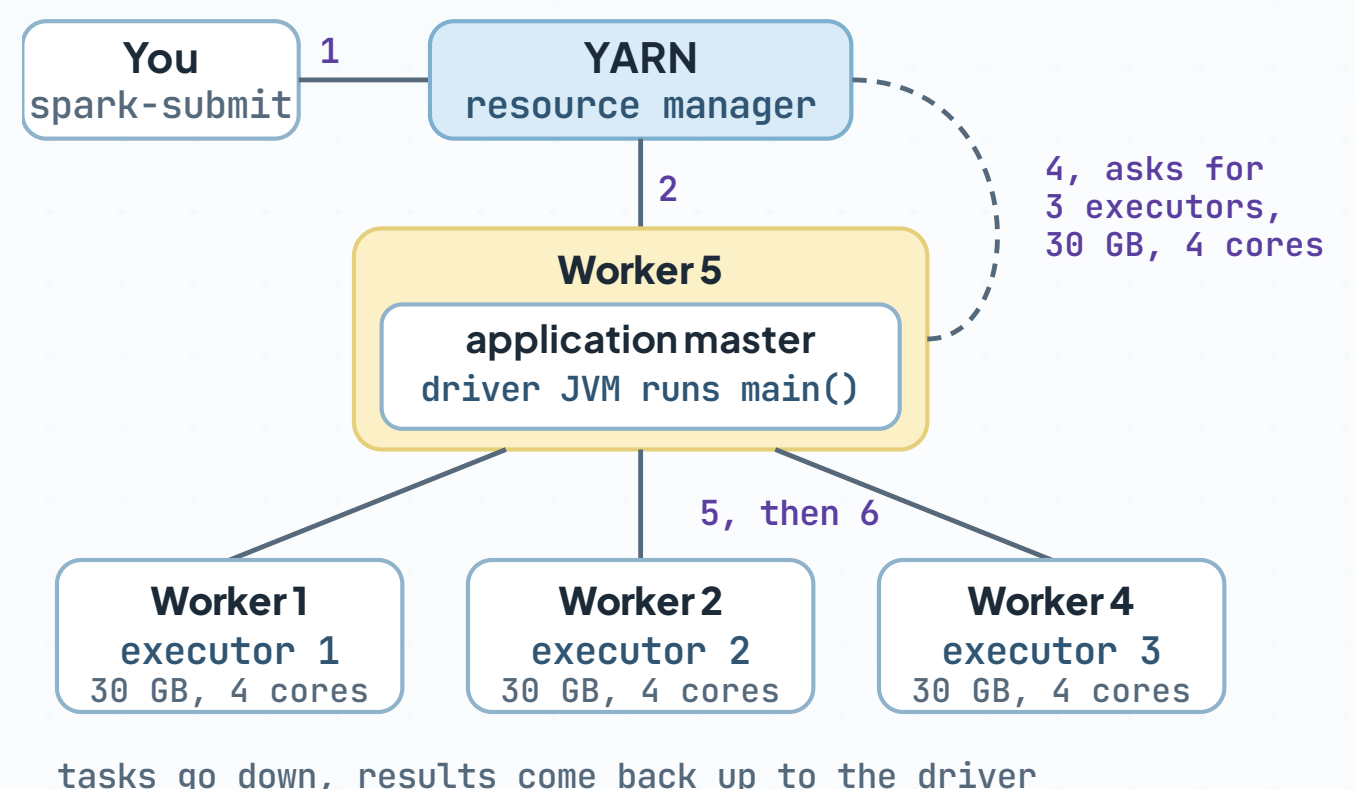


Three words first

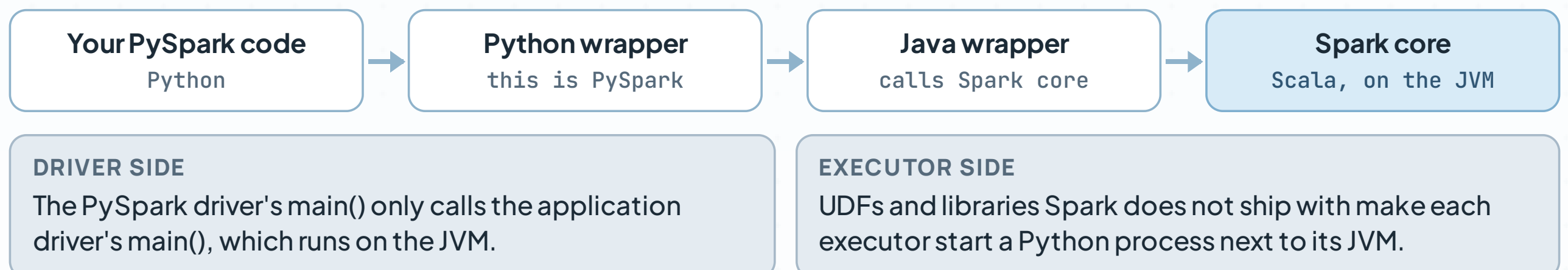
- 4 Cluster**
Machines on one network, used as one pool of compute.
- 5 Cluster manager**
Lives on the master node, hands out CPU and RAM. YARN or Kubernetes.
- 6 Container**
A fixed slice of one node's CPU and RAM. It cannot take more.

From spark-submit to results cluster mode

- 1** `spark-submit` sends your code and config to the YARN resource manager.
- 2** YARN creates the **application master** container on a random worker, here worker 5.
- 3** That container starts the **7 driver**, a JVM that runs your `main()`.
- 4** The driver asks YARN for what you requested: **3 executors, 30 GB and 4 cores each**.
- 5** YARN creates **8 executor** containers on workers 1, 2 and 4 and tells the driver where they are.
- 6** The driver schedules tasks. Executors run them and report results back.



9 Where PySpark fits the JVM does not speak Python

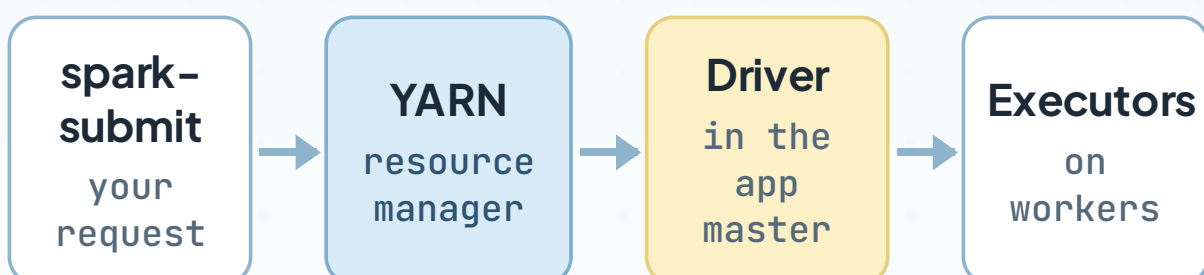


Where the driver runs, and what runs lazily.

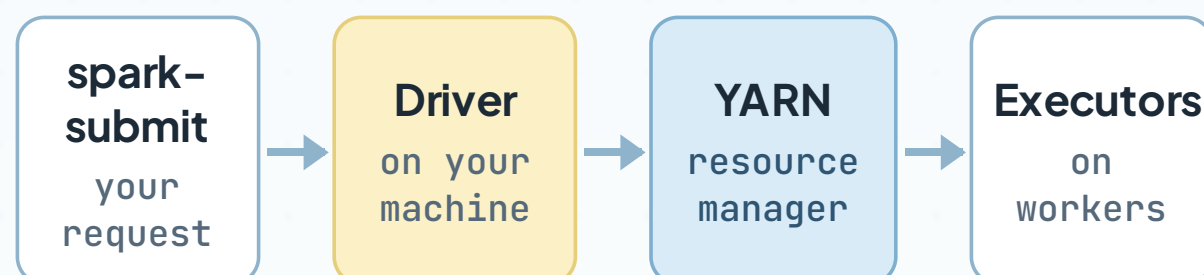
Concepts 10 to 16

TOPIC 3 Deployment modes

10 Cluster mode



11 Client mode



TOPIC 4 Transformations, actions, shuffle

12 Transformations are lazy

Each one only adds to the list of operations. Nothing runs yet.

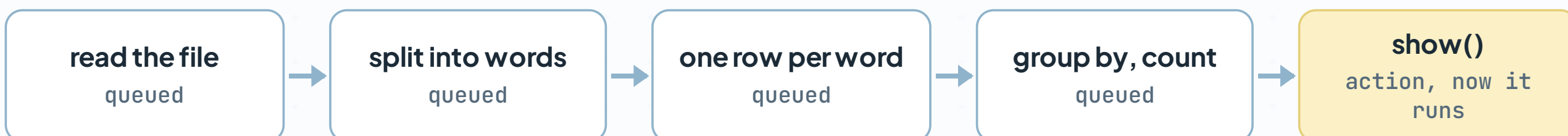
select filter withColumn groupBy

13 Actions run the list

One action triggers the whole computation that was queued before it.

collect show count save write

14 Lazy evaluation the word count, queued then run



15 Narrow vs wide, and 16 the shuffle

NARROW

No shuffle

One partition in, one partition out. filter, select, withColumn.

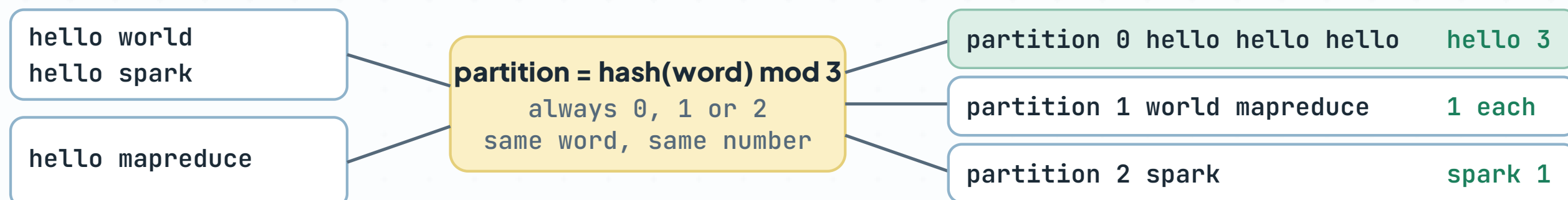
WIDE

A shuffle

Rows move across the cluster so equal keys meet on one partition. groupBy, repartition.

before: words wherever they were read

after: same word, same partition



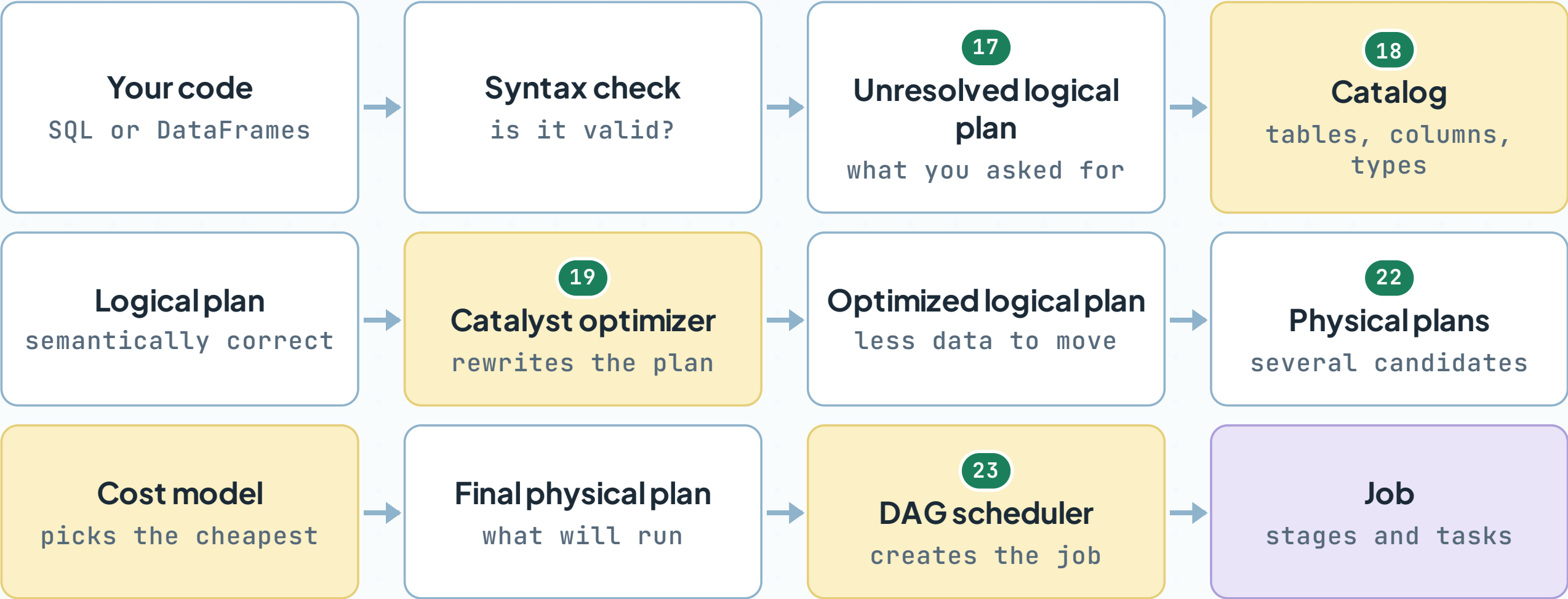
LEARNING

Every hello gets the same hash, so the count becomes a local sum. The move itself is the shuffle, the expensive part.

How the driver turns your code into a plan.

Concepts 17 to 23

From code to job three rows, left to right



The catalog check

```
SELECT A, B FROM table_A
```

Does `table_A` exist? Are `A` and `B` its columns? Syntax made the query valid; the catalog makes it correct.

The cost model and the DAG scheduler

One optimized plan can run in several ways. The cost model picks the cheapest physical plan; the DAG scheduler turns it into a job of stages and tasks for the executors.

THREE QUESTIONS THE DRIVER ANSWERS BEFORE ANY TASK LEAVES IT

- Is the code valid? The syntax check. Pass it and you have the unresolved logical plan.
- Do these tables and columns exist? The catalog check. Pass it and the plan becomes the logical plan.
- What is the cheapest way to run it? Catalyst rewrites the plan, then the cost model picks one physical plan.

Two Catalyst optimisations worth knowing less data ever reaches the executors

20 FILTER PUSHDOWN

WHERE city = 'Boston'

The filter runs at the data source. Executors never receive the rows they would throw away.

21 PROJECTION PUSHDOWN

SELECT name, age FROM (SELECT * ...)

Only name and age are read, even though the inner query says select star.

Jobs, stages and tasks, on one piece of code.

Concepts 24 to 26

The code, as steps sales data, one parquet file

1	read the parquet file	narrow
2	filter amount > 1500	narrow
3	repartition(3)	wide
4	select region, amount	narrow
5	withColumn tax, from amount	narrow
6	groupBy region	wide
7	agg sum(amount), sum(tax)	narrow
8	withColumn timestamp	narrow
A	collect()	action

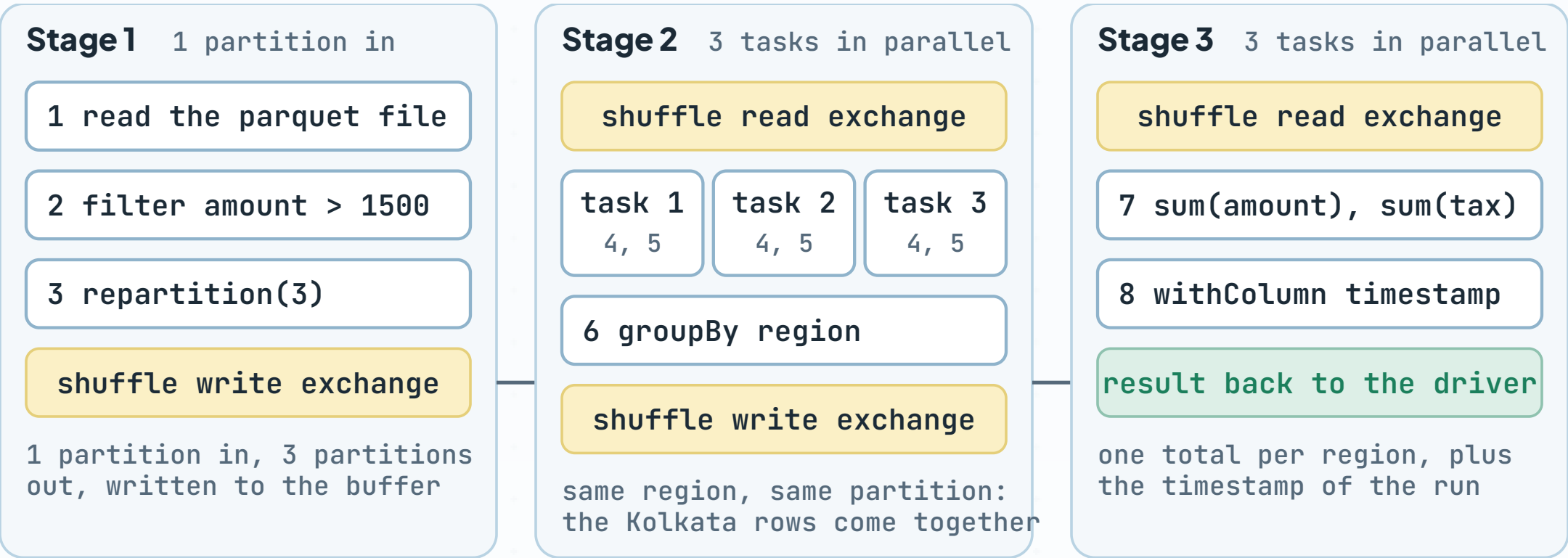
Three rules

RULE 1
24 An action creates a job
collect() creates one job that runs all eight steps.

RULE 2
25 Stages are cut at shuffles
n wide transformations given n + 1 stages. Two here, so three stages.

RULE 3
26 Partitions become tasks
One task per partition per core. Three partitions, three parallel tasks.

Stage by stage the exchange is the buffer shuffled data is written to and read from



Steps 4 and 5 run on the three partitions at the same time. Nothing depends on another partition until the groupBy, which is why the stage ends there.

WHY WIDE

repartition(3)

One partition in, three out. Rows have to move, and moving rows is a shuffle.

WHY WIDE

groupBy region

Every row of one region must meet on one partition before it can be summed.

Inside an executor's memory.

Concepts 27 to 30

The executor container what YARN created on workers 1, 2 and 4

